

IssueSpec: A Framework for Structured Review-to-Issue Translation

Anonymous Author(s)

Abstract

App developers daily receive millions of noisy reviews, reporting bugs, feature requests, performance issues, and device-specific problems. Manual review and triage cannot scale, and defect trackers such as JIRA require structured fields (steps to reproduce, severity, and component) rather than free text. Classifiers predict an issue label like “bug” or “feature request” but leave the remaining structured fields unaddressed. Response generators emit user-facing replies without explicitly identifying the underlying defect. Agentic software-engineering tools generate code patches from well-formed issue specifications, not from noisy review text. Knowledge graph tools cluster reviews by topic and produce free-form summaries rather than typed structured fields. Reinforcement Learning from Human Feedback (RLHF), which aligns Large Language Model (LLM) outputs with human preferences via pairwise comparisons, has been applied to general-safety settings but not to the multi-constraint compliance required for structured review-to-issue translation. We propose **IssueSpec**, a framework for structured review-to-issue translation built around a typed Intermediate Representation (IR) that converts each noisy review into one of five predefined report types (Zimmermann bug reports, Cohn user stories, ISO/IEC 25010 non-functional categories, Nielsen usability heuristics, and device-OS compatibility matrices). The framework combines knowledge-graph clustering for contextual grouping, a verified-anchor procedure for noise correction, the SPECov metric for faithfulness validation, and a Constrained Markov Decision Process (CMDP)-based alignment trainer for downstream response generation under quality and compliance constraints. We evaluate the framework on 215,583 reviews from 58 Android apps; on a stratified 28-spec subsample, it achieves 3.89/5 on a five-dimension issue-specification rubric. When these validated IssueSpecs are used as input to Retrieval-Augmented Generation (RAG), human-rated response quality improves by +2.36 Likert points over retrieval alone (5-point scale, $n = 100$ paired comparisons from a 400-row blinded rating set, $p < 0.001$), consistent across four LLMs from three model families. We release the IssueSpec schema, large-scale benchmark dataset, SPECov faithfulness metric, verified-anchor resource, and a CMDP-based RLHF testbed to support future research on structured information extraction and constrained generation.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; Reinforcement learning; • **Information systems** → *Information extraction*; • **Software and its engineering** → *Software maintenance tools*.

Keywords

App Reviews, Knowledge Graph, Intermediate Representation, Retrieval-Augmented Generation, RLHF, Software Engineering, Issue Tracking

1 Introduction

App stores are where users talk back to developers. A single popular app pulls in thousands of reviews a week, and the global stream runs into millions a day [19, 22, 42]. These reviews carry the signals developers need to act on: bug reports (a crash on Samsung S8 after an update), feature requests (dark mode), performance complaints (battery drain), and regression notes (voice search no longer works). The information is real, but it shows up as messy free-form text, usually without the details required to reproduce a bug or scope a new feature.

To turn any of this into work, developers rely on defect trackers such as JIRA or GitHub Issues. Those tools, though, can't work with raw text; they need structured fields such as issue type, steps to reproduce, severity, and affected component [9, 66]. Filling those fields by hand does not scale with the volume modern apps face. Years of work on automated review pipelines has helped with the surrounding tasks like classification, summarization, and response generation, but no system actually produces the structured report a developer can paste directly into a tracker. Dąbrowski et al.'s nineteen-year survey of the field [19] confirms the same boundary.

Prior work has tackled several pieces of this problem separately. Classifiers [10, 17, 42, 61] take a review and predict an issue label such as bug or feature request. Response generators [22, 23, 41] produce a polite reply for the user. Agentic software-engineering tools such as SWE-Agent [63] and RepairAgent [6] take a clean GitHub issue and generate a code patch. Knowledge-graph methods [15, 36, 64] cluster reviews into themes or surface frequent complaints. Each of these has matured over the past decade and works well within its own scope.

But none of these reaches the developer-ready issue report a defect tracker needs. The Maalej-Nabil classifier taxonomy [42] and its successors [10, 17, 35, 58, 61] have settled at an F1 ceiling around 0.75–0.85, a label-quality limit (not a model-capacity one) per Dąbrowski et al.'s [19] nineteen-year survey. The label is only the first piece of what a tracker entry needs; the remaining fields (steps to reproduce, severity, affected component) have to come from somewhere, and classification-only pipelines leave that downstream step to manual triage. Response generators like RRGGen [22] and CoRe [23] address the customer-communication side of the workflow, but the actual bug behind the review is never written out in the structured form a tracker needs, so even a perfect reply system does not give developers something they

can paste into JIRA. Agentic SE tools assume a well-formed bug report or GitHub issue as the agent’s starting state, built into the SWE-bench protocol [33] where each instance is a (GitHub issue, code repository) pair; constructing such an issue from raw review text is explicitly outside the protocol, leaving the upstream gap unaddressed.

Knowledge-graph methods stop at intermediate outputs rather than typed fields: ReviewGraph [15], KGCPN [64], and Keertipati et al. [36] each output rating predictions, summaries, or feature rankings, none of which a defect tracker can ingest directly. RLHF has matured into the standard approach for aligning LLM outputs with human preferences (DPO [52], KTO [20], PPO [57]), and Safe-RLHF [13] extends the framework to multi-objective settings under a Constrained MDP for general LLM safety (harmfulness, dishonesty, bias). The developer-relations setting poses a different question entirely: how to align responses under a domain-specific constraint family (truthful promises, no information leakage, professional tone, legal-safe wording) rooted in business and legal exposure. To our knowledge, no published RLHF formulation has been adapted to this constraint set.

This work pursues three aims. First, we develop a framework that turns noisy app reviews into structured, taxonomy-grounded issue specifications a defect tracker can consume directly. Second, we build a response generator that uses these specifications as input, producing replies that reference the actual underlying defect rather than generic templates. Third, we close the loop with a dual-objective alignment trainer that jointly optimizes for response helpfulness and the domain-specific compliance constraints of the developer-relations setting.

These aims map to three research questions.

RQ1 (Translation Quality). Can a typed IR, generated via a three-layer framework (knowledge-graph construction, hierarchical clustering, standardized schema mapping), produce issue specifications that match or exceed the substantive completeness of human-written GitHub issues?

RQ2 (Downstream Load-Bearing). Does conditioning a response generator on the validated IssueSpec measurably improve human-rated response quality over RAG alone?

RQ3 (Dual-Objective Alignment). Can a CMDP-grounded RLHF loop progressing through KTO, DPO, and Constrained PPO jointly optimize response helpfulness and policy compliance for the developer-relations setting?

To address these challenges, we propose **IssueSpec**, a framework for structured review-to-issue translation. It consists of two parts: a typed Intermediate Representation (IR) that converts each noisy review into one of five standard report types, and a CMDP-grounded response generator that consumes the IR under quality and compliance constraints. Figure 1 shows the full five-stage pipeline.

The main contributions of this work, each mapped to the gaps above, are:

- **We propose IssueSpec**, the first typed Intermediate Representation (IR) for review-to-issue translation: a typed dictionary routed by issue class to one of five SE-taxonomy templates with verified standards-body provenance [12, 31, 34, 47, 66].
- **We introduce SPEC-COV**, a stand-alone extractive-coverage faithfulness metric grounded in news-summarization practice [26, 38, 44], together with a verified-anchor pipeline using confident learning [48] that lifts Cohen’s κ from 0.163 to 0.592 with 88.66% independent classifier endorsement of corrections across 40,291 cases.
- **We design** a three-layer aspect-grounded knowledge graph that produces and prioritizes IssueSpecs via dependency-parsed aspect mining [30], sentence-embedding-based clustering [45, 46, 53], and PageRank centrality [7, 49]; the top-five aspects cover 74% of all review-aspect mentions, roughly $10^3\times$ concentration in the developer’s reading order.
- **We evaluate** the framework end-to-end on 215,583 reviews from 58 Android apps: IssueSpec achieves 3.89/5 on a five-dimension issue-specification rubric, and adding it to RAG lifts human-rated response quality by +2.36 Likert points on 400 paired ratings ($p < 0.001$), while retrieval alone is statistically indistinguishable from a no-context baseline; a four-LLM cross-family replication (Section 5.1) rules out single-model bias.
- **We release** a CMDP-grounded RLHF testbed (KTO [20] to DPO [52] to Lagrangian Constrained PPO [13, 57]), formulated as a Constrained Markov Decision Process [1, 13] that separates quality reward from compliance constraints, together with a benchmark dataset of 100 stratified clusters, a 490-review classification gold standard, 400 paired ratings, and a 99-review three-rater Krippendorff $\alpha = 0.451$ subsample.

Table 1 maps each contribution to its measurable headline result and released artifact. Figure 1 shows the five-stage pipeline.

2 Related Work

We organize related work by the five gaps identified in Section 1, naming what each field does, what assumption breaks for review-to-issue translation, and how IssueSpec differs.

2.1 App Review Classification and Mining

Maalej and Nabil [42] established the canonical seven-class taxonomy for app reviews. Subsequent systems, AR-Miner [10], CLAP [61], SURF [17], BERT/RCNN hybrids [35, 58], and per-aspect sentiment analysis [27], refined classification accuracy and added aspect-level granularity. Dąbrowski et al. [19] surveyed nineteen years of such work and concluded that label quality, not model capacity, is the F1 ceiling at 0.75–0.85, an observation we corroborate in Section 5.1. *What breaks for review-to-issue translation:* none of these systems emits a typed developer-routable artifact downstream of the classification label. IssueSpec

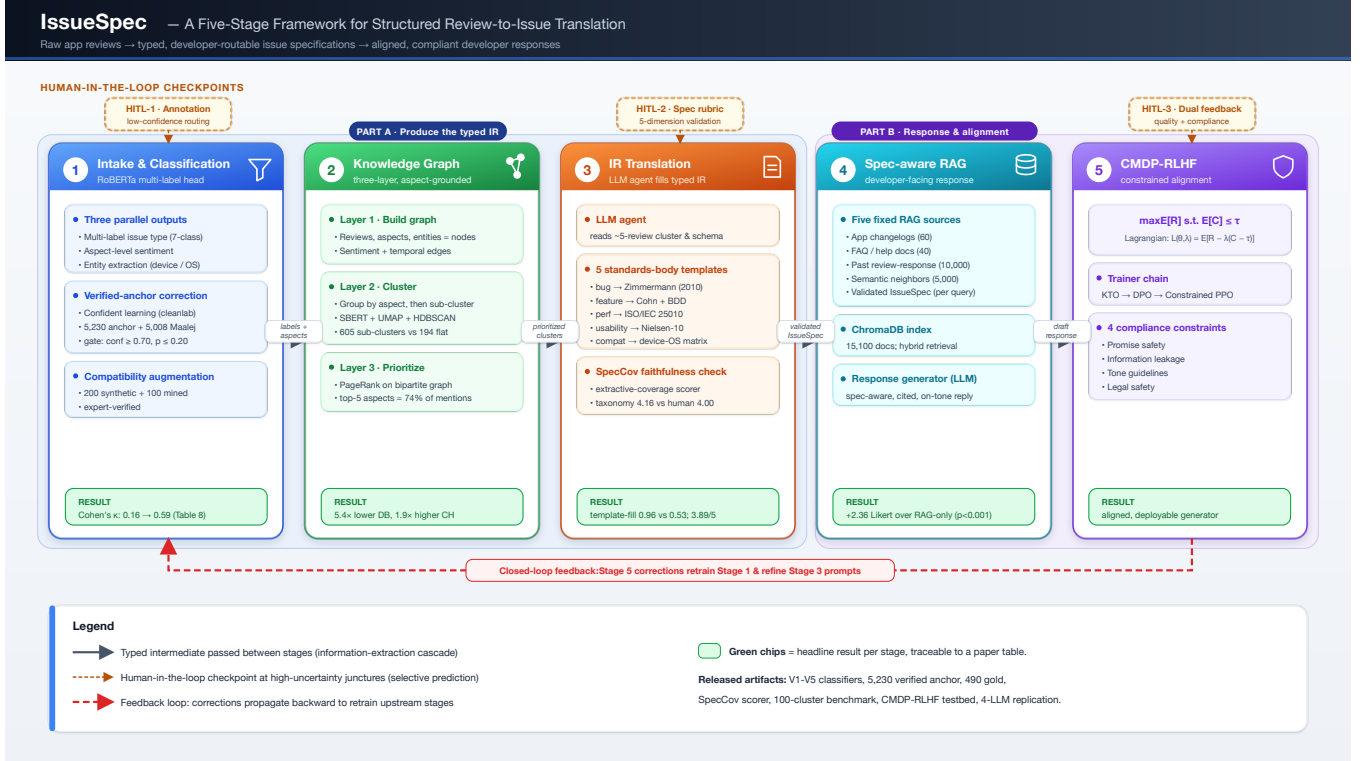


Figure 1: The IssueSpec five-stage pipeline. Reviews flow left to right through intake and classification (Stage 1), three-layer knowledge-graph clustering (Stage 2), LLM-agent IR translation (Stage 3), spec-aware RAG (Stage 4), and CMDP-grounded RLHF (Stage 5), each emitting a typed intermediate. Yellow checkpoints mark human-in-the-loop review under selective prediction [5, 24]; the red loop propagates Stage 5 corrections upstream.

differs by routing the predicted class to one of five typed templates rather than terminating at the label.

2.2 Response Generation and Retrieval-Augmented Generation

RRGen [22] and CoRe [23] pioneered end-to-end neural response generation on (review, response) pairs without an explicit structured intermediate. Vanilla RAG [41] retrieves relevant past responses in a single pass. *Agentic RAG* variants, Self-RAG [4] and DEMONSTRATE-SEARCH-PREDICT (DSP) [37], introduce iterative critique-revise loops that, in principle, could close the structured-intermediate gap. *What breaks for review-to-issue translation*: none of these systems names the underlying defect, and our $n = 10$ feasibility study (Section 5.2) shows that agentic iteration without a typed intermediate adds modest gains (0.58→0.70 rule-based score) compared with the +2.36 Likert-point gain from adding the IssueSpec. IssueSpec differs by injecting the typed intermediate as a first-class RAG source alongside retrieved past responses.

2.3 Agentic Software Engineering

SWE-Agent [63], RepairAgent [6], and HyperAgent [51] consume well-structured GitHub issues to produce code patches

through multi-step planning and tool use. *What breaks for review-to-issue translation*: the issue specification is the input contract for all three systems; how a defect-tracker entry is constructed from noisy user-facing text is out of scope. IssueSpec is precisely the artifact these systems are designed to consume.

2.4 Knowledge Graphs for App Reviews and Software Engineering

Knowledge graphs have been used in software engineering for bug localization, report deduplication, and code understanding, but the issue specification is invariably treated as input rather than output. For app reviews specifically, ReviewGraph [15] learns KG embeddings for rating prediction, KGCPN [64] produces KG-based extractive summaries, and Keertipati et al. [36] apply graph centrality for coarse feature prioritization. *What breaks for review-to-issue translation*: none of these systems maps KG nodes to a typed, developer-actionable issue schema, and no prior KG work has been applied to the specific task of producing GitHub-issue-quality artifacts from review text. IssueSpec’s three-layer KG pipeline, structured-object layer, aspect-grounded

Table 1: Contribution-to-evidence-to-artifact map. Each contribution states a measurable headline result, the table or section that anchors it, and the named artifact released with this paper.

contribution	headline evidence	released artifact
IssueSpec	0.96 template-fill vs 0.53 GitHub (Table 7); 5-dim rubric 3.89/5 (Table 6)	type-routed schema (Table 3); SPECov extractive-coverage scorer
IR load-bearing	+2.36 full vs no_spec on 400 paired (Table 10); no_spec indistinguishable from prompt baseline	400 blinded paired ratings + 30-pair second-rater subset
verified anchor	κ 0.16→0.59 (Table 8); per-class F1 recovery (Table 9); 88.66% V5 endorsement on 40,291 cases	5,230-review verified anchor + cleanlab correction procedure + V5 classifier
KG prioritization	5.4× lower Davies-Bouldin (Table 11); top-5 PageRank aspects cover 74% of mentions; count-controlled A1b (§5.5)	three-layer KG construction + count-controlled A1b output
CMDP RLHF + benchmark	constrained-proxy +52% BLEU-1 over SFT base (§5.3); satisfied-at-init compliance scorer	CMDP formulation, KTO/DPO/constrained-proxy/CPPO trainers, benchmark (100 clusters, 400 ratings, 490 gold, $\alpha = 0.451$ subsample)

hierarchical clustering layer, and PageRank-prioritized issue-mapping layer, is, to our knowledge, the first KG architecture that emits a typed IR for review-to-issue translation.

2.5 RLHF and Constrained Optimization for Response Generation

DPO [52], KTO [20], and PPO [57] compress quality and compliance signals into a single scalar reward. Altman’s CMDP framework [1] separates these objectives, and Safe-RLHF [13] applies CMDP to general LLM safety with a domain-agnostic compliance set covering harmful, dishonest, and biased outputs. *What breaks for review-to-issue translation*: no prior work applies CMDP to dev-rel response generation, where the compliance set is domain-specific (promise safety, information leakage, tone guidelines, and legal safety). Our Stage 5 formulation differs by instantiating these four domain-specific constraints inside the CMDP Lagrangian and releasing a proof-of-concept training loop.

2.6 Cross-Category Positioning

Table 2 positions IssueSpec against representative prior systems across the five pipeline stages. Existing systems occupy at most two of the five stages; composing all five with a standardized issue schema, three theoretically-grounded HITL checkpoints, KG-centrality prioritization, and a CMDP-grounded RLHF formulation is the structural contribution of this work.

3 The IssueSpec Pipeline

The IssueSpec pipeline (Fig. 1) is a five-stage information-extraction cascade [28, 56]. Stages 1–3 produce the typed IR from raw reviews; Stage 4 uses it to drive spec-aware response generation; Stage 5 aligns the response generator via constrained RLHF. We describe each stage in turn, with the verified-by-citation taxonomy templates in Table 3.

3.1 Producing the IR

Stage 1: Intake, Classification, and Aspect Extraction. Stage 1 produces three outputs per review: a multi-label class from a RoBERTa head fine-tuned on Maalej [42] and Guzman [27] (reviews often mix concerns, like a bug report that also asks for a feature); aspect-level sentiment [27] so “great UI but terrible battery” is kept as separate pairs rather than averaged to a star rating; and entity extraction for devices, OS versions, and named features, which the Stage 3 device-OS and bug-report templates need as typed fields. Low-confidence reviews route to HITL-1 under selective-prediction theory [24]: when the classifier cannot decide, the expert does, and the correction feeds the next training round.

LLM annotation noise is the main challenge here. On the 5,041-review praise stratum, 25% of LLM labels are wrong, concentrated on minority classes. Discarding them wastes annotation; using them as-is poisons training. We apply confident learning [48] against a verified anchor of 5,230 expert-labeled reviews plus 5,008 from Maalej [42], gated by anchor confidence ≥ 0.70 and LLM probability ≤ 0.20 ; this keeps the good labels and corrects only the wrong ones. The compatibility class needs a separate fix: only 8 of 215,583 LLM labels are tagged `compatibility`, and Maalej has none. We seed it with 200 synthetic reviews from device, OS, and screen templates [34], plus 100 mined from RRGGen via compatibility keywords, each checked by the lead author before training.

Stage 2: Three-Layer Knowledge Graph. Stage 2 organizes reviews into an aspect-grounded knowledge graph: build, cluster, prioritize. The first layer builds the graph: each review is a node linked to its aspects and entities by sentiment and temporal edges. A graph beats a flat list because reviews share aspects across users, and that co-occurrence is what later layers cluster and rank on. Aspects come from dependency-parse noun phrases following Hu and Liu [30], pruned by frequency and centrality to drop one-off terms.

The second layer clusters reviews first by aspect, then sub-clusters within each aspect using sentence embeddings [53], UMAP [46], and HDBSCAN [45]. Aspect-first keeps each sub-cluster on one theme: a flat clusterer mixes “battery on Samsung” with “battery in dark mode” because both share lexical content. The result is 605 aspect-grounded sub-clusters with mean size 3.0, against 194 from a flat baseline. The third layer prioritizes via PageRank [7, 49] on the bipartite review-aspect graph, ranking aspects by cross-review prevalence so developers see the most-affected first. Top three: `ad` (0.040), `phone` (0.011), `battery` (0.007). Each

Table 2: Cross-category positioning. Existing systems compose at most two of the five pipeline stages. IssueSpec is the first to compose all five with a standardized issue schema, three HITL checkpoints grounded in selective prediction theory, KG-centrality prioritization, and a CMDP-grounded RLHF formulation. “—” denotes a stage absent from that system.

System	Classify	Cluster	IssueSpec	Response	RLHF
AR-Miner [10]	filter	LDA topic	—	—	—
CLAP [61]	rating	DBSCAN	—	—	—
SURF [17]	intention	LDA	—	summary	—
RRGen [22]	—	—	—	seq2seq	—
CoRe [23]	—	—	—	context-aware retrieval	—
Maalej [42]	7-class	—	—	—	—
ReviewGraph [15]	rating	KG	—	—	—
SWE-Agent [63]	—	—	consumes (not produces)	—	—
Safe-RLHF [13]	—	—	—	general	dual-CMDP (general)
IssueSpec (ours)	V5 corrected	KG + PageRank	type-routed schema	spec-aware RAG	dual-CMDP (dev-rel)

cluster maps to the fixed schema in Table 3 with a centrality-derived priority score.

Stage 3: Review-to-Issue Translation. Stage 3 instantiates the IssueSpec. An LLM agent reads the standardized cluster schema and writes a GitHub-issue-quality specification using the type-routed templates in Table 3. We use an LLM rather than rule-based extraction because reviews vary too much for fixed rules and the agent has to reason across a 5-review cluster. We route by issue type because a bug report needs reproduction steps, a feature request needs acceptance criteria, and a performance complaint needs latency or memory numbers; a shared template would be incomplete or padded. The five templates are Zimmermann bug reports [66], Cohn user stories [12] with BDD acceptance criteria [62], ISO/IEC 25010 non-functional categories [31], Nielsen heuristics [47], and device-OS matrices [34]. Each is grounded in its standards-body citation because developers already recognize these formats.

The output passes through HITL-2: a domain expert scores each spec on a five-dimension rubric (completeness, accuracy, actionability, specificity, clarity, each 1–5), following the LLM-as-judge calibration convention of Gilardi et al. [25], Pangakis et al. [50], and Zheng et al. [65]. Five dimensions, not one, because a single quality number conflates completeness with clarity and the LLM cannot tell which one to fix on retry. Any spec failing on any dimension goes back to the LLM with dimension-level feedback for a second pass.

SPECov faithfulness metric. We release SPECov, an extractive-coverage scorer that measures substantive-token overlap ($\text{length} \geq 5$, stop-word filtered) with quoted-string and coverage bonuses. Bands follow news-summarization conventions [26]; the metric targets the extractive-coverage sub-type of faithfulness [26, 44], a known lexical correlate of factual consistency [38] (the remaining hallucination sub-types in Ji et al. [32] are out of scope). Across 320 specs, the LLM-with-taxonomy condition averages 4.16, LLM-free-form 3.33, raw concatenation 5.00 (trivial upper bound from verbatim copying), and human reference 4.00. On the standard 5-point faithfulness scale, scores ≥ 4 are conventionally read as substantively faithful in news-summarization practice [38, 44]; both the taxonomy condition (4.16) and the human reference (4.00) sit in this band. The taxonomy condition matches or exceeds

the human band and correctly penalizes the dominant hallucination mode, introducing devices, OS versions, or components absent from the source.

3.2 Using the IR in Response Generation

Stage 4 is a RAG layer over five fixed sources (15,100 documents total in the released ChromaDB index): (i) app changelogs and release notes (60 docs); (ii) FAQ and help documentation (40); (iii) 10,000 past review-response pairs from RRGen [22]; (iv) 5,000 semantically similar past responses; and (v) the validated IssueSpec from Stage 3, injected per query because specs are generated per cluster rather than pre-indexed. These five were chosen to give the generator the answer (changelogs, FAQ), the voice (past responses), the closest precedent (semantic neighbors), and the structured intent (IssueSpec) without overlap.

Vanilla versus agentic RAG. The headline 400-pair evaluation runs single-shot vanilla RAG [41] so the IssueSpec’s contribution stays isolated from agentic orchestration. We additionally compare against the closest agentic-RAG baselines, Self-RAG [4] and DSP [37], on a 10-review feasibility sample (stratified, two per issue type), using Qwen2.5-3B as critic and reviser with up to two iterations. The agentic variant scores 0.70 on the rule-based scorer versus 0.58 for vanilla single-shot (+0.12); the citation rate rises from 0% to 60%. Both gains are dwarfed by the +2.36 Likert-point gain from adding the IssueSpec (Section 5.2): the structural intermediate carries the work, not the iteration.

The four-condition design in Section 5.2 isolates the load-bearing source: the prompt-baseline adds dev-rel system instructions atop an rrgen-style baseline, `no_spec` adds RAG, and `full` adds the validated IssueSpec. This setup lets us read off whether RAG alone moves the needle (it does not) and whether the structured IR does (it does).

3.3 Aligning the IR-Aware Generator

Stage 5 frames alignment as a CMDP [1, 13]: maximize a quality reward subject to a compliance constraint,

$$\max_{\theta} \mathbb{E}_{\pi_{\theta}}[R_{\text{quality}}] \quad \text{s.t.} \quad \mathbb{E}_{\pi_{\theta}}[C_{\text{compliance}}] \leq \tau,$$

Table 3: IssueSpec standardized issue schema. Each issue type follows the dominant industry or HCI standard; templates are verified against their standards-body citations, producing GitHub-issue-quality artifacts a defect tracker can consume directly.

Issue type	Template (citation)	Justification
bug_report	Zimmermann [66]: title, severity, steps_to_reproduce, expected, actual, component	Standard in GitHub Issues, JIRA, defect trackers; Chaparro et al. [9] confirm these are the most-requested bug-report fields
feature_request	Cohn user-story [12] (As-a / I-want / So-that) + behavior-driven development (BDD) acceptance criteria [62]	Dominant industry format (Agile, Scrum, SAFe); BDD standardizes testable outcomes
performance	ISO/IEC 25010 [31] performance-efficiency sub-characteristics: time behavior (speed, responsiveness), resource utilization (memory, battery), capacity (scalability)	International standard for software-quality NFR taxonomy; the parenthesized terms map ISO sub-characteristics to mobile-app vocabulary
usability	Nielsen-10 heuristics [47] (visibility, match-real-world, user control, etc.)	Most-cited usability evaluation framework in human-computer interaction
compatibility	Device-OS matrix (devices $\times$ OS versions) [34]	Device/OS fragmentation is the canonical mobile-QA reporting axis

with the Lagrangian relaxation

$$\mathcal{L}(\theta, \lambda) = \mathbb{E}_{\pi_{\theta}}[R - \lambda(C - \tau)]$$

collapsing gracefully to single-objective optimization whenever the constraint is already satisfied. We use a CMDP rather than a scalar reward because a dev-rel response can be high-helpfulness and still out of compliance, like a polite promise the team cannot deliver or a leaked roadmap detail, and a weighted sum hides exactly the trade-off the constraint is meant to expose. The compliance scorer tracks four domain-specific dimensions absent from prior general-safety RLHF [13]: promise safety (no commitments the team cannot keep), information leakage, tone guidelines, and legal safety. The trainer is Constrained PPO [13, 57] with a KL-regularized reference policy that keeps updates from drifting too far from the supervised baseline. KTO [20] and DPO [52] are reported as small-scale single-objective comparators in Section 5.3; together they bracket the preference-data spectrum, KTO needing only binary feedback and DPO needing pairwise.

HITL-3 supplies two parallel feedback streams. *Quality* feedback (helpfulness, specificity, empathy, accuracy, actionability) is the reward; *compliance* feedback (promise safety, leakage, tone, legal) is the constraint. Feedback also propagates backward: classification corrections retrain Stage 1, rubric scores refine Stage 3 prompts, and response scores drive the RLHF updates, closing the Stage 5→Stage 1 loop in Fig. 1.

4 Experimental Setup

4.1 Dataset, Deduplication, and Annotation Provenance

We use the RRGGen corpus [22]: 310,031 review-response pairs from 58 Android applications. After exact-text deduplication that removes 94,448 duplicate reviews (−30.5% of the raw corpus), 215,583 unique reviews form the working corpus, labeled across the seven Maalej [42] classes. Evaluation additionally uses a 490-review expert gold standard (70 per class $\times$ 7 classes) and 100 stratified clusters for Stage 3 evaluation.

Annotation provenance (full corpus, 215,583 reviews). Of the working corpus, 79.49% of labels come from

the V2 LLM annotator with no override, 18.08% are cleanlab-corrected against the verified anchor, and 2.43% are directly human-verified by the lead author. The lead-author-versus-V2-LLM deviation is 25%, replicated independently across two strata: the 5,041-review praise stratum and the 5,230-row human-verified anchor subset (Table 4). This $\approx$ 25% deviation rate, an order of magnitude higher than i.i.d. label-noise assumptions, motivates the confident-learning procedure of Stage 1.

4.2 Annotation Methodology

Four kinds of annotator contribute, following the multi-annotator convention of Gilardi et al. [25] and Pangakis et al. [50] who treat LLM annotators as independent raters for inter-rater statistics.

(i) **Lead author (domain expert).** Verified 5,230 reviews across minority strata (approximately thirty person-hours), wrote 20 reference IssueSpecs, labeled the 490-review classification gold blinded to V2 LLM, and scored 400 Stage 4 pairs on quality (1–5), specificity (1–5), and helpful (Y/N) with method IDs hidden until aggregation.

(ii) **Second human rater.** Independently re-rated 30 paired (no_spec, full) responses on the same rubric, blind to the lead author’s scores, to calibrate HITL-3.

(iii) **LLM-as-judge (Qwen2.5-3B-Instruct).** Scored 28 IssueSpecs on the five-dimension rubric and 50 sub-clusters on the Y/P/N purity rubric [65], both calibrated against the lead author.

(iv) **Corpus-labeling LLMs.** The V2, cleanlab-V2, and V5 LLMs are treated as independent raters for inter-rater statistics.

Table 4 consolidates agreement and disagreement across all pairs.

Ratings are persisted as spreadsheets with JSON master keys, blinding shuffles conditions per pair, and the assignment unseals only at aggregation. The three-rater Krippendorff α uses the lead author plus two Qwen2.5-3B prompts (concise and chain-of-thought).

4.3 Multi-LLM Bias Control

A single-LLM headline raises a structural-bias concern: results from any one model may not transfer across families.

Table 4: Annotator-vs-LLM and inter-annotator deviation. The $\approx 25\%$ deviation is replicated on two independent strata; the κ -progression reaches moderate agreement [40]; 88.66% V5 endorsement is the strongest cross-classifier signal; LLM-judge calibration Δ bounds judge bias.

Annotator pair	sample	agreement / deviation
Lead-author vs V2 LLM	10,271	75% / 25% mislabeled
Lead vs V2 / cleanlab / V5	490 gold	κ : 0.163 $\rightarrow$ 0.333 $\rightarrow$ 0.592
V2 LLM vs V5; cleanlab vs V5	215,583	71.96% / 86.77% agree
cleanlab vs V5 endorsement	40,291	88.66% support
Lead vs 2nd rater (Stage 4)	30 paired	86.7% within-1; 22/30
3-rater (lead + 2 Qwen)	99 reviews	Krippendorff $\alpha = 0.451$
Lead vs Qwen-judge (purity, 50 / 28 rubric)		$\Delta = -0.035 / +0.23$

We therefore replicate Stage 3 across four LLMs drawn from three distinct families, Anthropic Claude (Opus 4.7), Meta Llama-3.3-70B-Instruct (via Groq), and Alibaba Qwen2.5-{3B, 1.5B}-Instruct, with Stage 4 replicated on a Claude-vs-Qwen-3B subset, and check that cross-family rank order is preserved (Table 5). Frontier-family replication on GPT-4o and Gemini-1.5-Pro is queued.

4.4 Experimental Design

We pre-register three experiments, one per research question:

Experiment 1 (RQ1, translation quality). Five conditions are compared on the same 100 stratified clusters: (a) LLM with taxonomy grounding (the full Stage 3 system), (b) LLM without taxonomy grounding (free-form), (c) raw review summary (a lower bound that concatenates the cluster’s reviews), (d) lead-author reference, and (e) human-written GitHub issues mined from three open-source Android projects (the practitioner-quality upper bound). Each spec is scored on the five-dimension rubric. Significance is reported via paired Wilcoxon signed-rank tests on rubric scores between conditions.

Experiment 2 (RQ2, coupled vs. uncoupled). The same 100 clusters drive four response-generation conditions: (a) an rrgen-style baseline (review only), (b) a prompt-baseline (CoRe-style with dev-rel prompt and lightweight entity extraction), (c) RAG without IssueSpec, and (d) the full system with IssueSpec. Evaluation uses automatic metrics (BLEU, ROUGE-L, BERTScore) and 400 blinded paired human ratings (quality 1–5, specificity 1–5, helpful Y/N), with the lead author as primary annotator and a second rater on a 30-pair subset. Statistical significance is checked simultaneously by paired Wilcoxon, Friedman with Nemenyi post-hoc [16], Bradley-Terry, and McNemar at Bonferroni-corrected $\alpha = 0.05$. Effect sizes follow Romano et al. [54] and Cliff [11].

Experiment 3 (RQ3, dual- vs. single-objective RLHF). Five policies run on a distilGPT2 proof-of-concept (82M parameters, 400 SFT samples, 296 KTO pairs, 100

Table 5: Multi-LLM evidence summary across three model families (Anthropic, Meta, Alibaba). Cross-family rank preservation across Stage 3 and Stage 4 is the structural-bias control; LLM-as-judge calibration Δ vs the lead author bounds judge bias.

Use	Model (family)	Result
Stage 3 generate	Claude (Anthropic) / Llama-3.3-70B (Meta) / Qwen-3B (Alibaba) / Qwen-1.5B (Alibaba)	0.96 / 0.80 / 0.74 / 0.48
Stage 4 generate	Claude / Qwen-3B	0.61 / 0.54
5-dim rubric judge	Qwen-3B vs lead-author	3.89/5; $\Delta = +0.23$
Y/P/N purity judge	Qwen-3B vs lead-author	0.625 vs 0.66; $\Delta = -0.035$

DPO pairs): SFT-base, single-objective KTO [20], single-objective DPO [52], a constrained-proxy variant, and dual-objective Lagrangian Constrained PPO [13, 57]. We measure head-to-head BLEU, ROUGE-L, and BERTScore on a 100-review test set, plus the rule-based safety-violation rate.

5 Results

5.1 Is the IR Producible at Scale?

Under strict content-validity criteria (at least three reproduction steps with action verbs, descriptions ≥ 30 words, and a full As-a/I-want/So-that triple for user stories), the LLM-with-taxonomy condition achieves a substantive template-fill rate of 0.96 (Table 7). Free-form LLM reaches 0.69, the same as the lead-author reference set ($n = 20$). Raw concatenation lands at 0.34, and real GitHub issues mined from three open-source Android projects reach 0.53. The LLM-vs-GitHub gap reflects coverage rather than content quality: the LLM is prompted to fill every field, whereas humans leave optional fields blank. A very-strict sensitivity sweep preserves the ordering, Claude drops from 0.97 to 0.71, with the Claude-vs-GitHub gap shifting only -0.085 .

Cross-LLM replication (bias control). Table 5 reports four uses across three model families. Llama-3.3-70B reaches 0.80 template-fill, Qwen2.5-3B 0.74, and Qwen2.5-1.5B 0.48 under the same strict criteria; the templated $>$ free-form $>$ raw ordering holds across all four models with capability-scaled magnitudes. On two per-class structural-compliance checks, Llama-3.3-70B attains complete template compliance on the stratified sample: all 29 bug specs include three or more reproduction steps with action verbs (vs. Claude’s 73%), and all 30 feature specs include the As-a/I-want/So-that triple (vs. Claude’s 97%). These measure structural template compliance (does the field exist with the required form?), not content correctness, content quality is reported separately under the five-dimension rubric. The structural-bias control that matters here is cross-family rank preservation: the IssueSpec contract transports across families rather than depending on a single proprietary generator.

Table 6: Experiment 1 five-dimension rubric (Qwen-judge, 1–5 scale). LLM-with-taxonomy condition ($n = 28$, stratified 6/issue-type) vs lead-author reference set ($n = 20$). The LLM-as-judge protocol follows the calibration-against-reference convention of [25, 50, 65]. Lead-author specs are higher on actionability and lower on coverage dimensions: an expert-vs-LLM contrast confirming the judge is not biased.

dimension	LLM + taxonomy	lead-author ref
completeness	3.79	3.05
accuracy	4.39	3.75
actionability	3.71	4.70
specificity	4.07	3.80
clarity	3.50	3.00
overall mean	3.89	3.66

Table 7: Experiment 1 substantive template-fill (strict criteria). The 0.96 vs 0.53 LLM-vs-GitHub gap is coverage, not content quality; the strict-vs-very-strict sweep preserves ordering. The exact-zero cells in bugs:repro and features:story columns for (b), (c), (d), and (e) are structural: free-form LLM, raw concatenation, lead-author, and human-GitHub specs do not emit the template-specific reproduction-step list or As-a/I-want/So-that triple at all when not prompted for them, so the strict-fill rate is zero by construction.

condition	strict	bugs:repro	features:story
(a) LLM + taxonomy	0.96	0.73	0.97
(b) LLM free-form	0.69	0.00	0.00
(c) raw concat	0.34	0.00	0.00
(d) lead-author ref ($n = 20$)	0.69	0.00	0.00
(e) human GitHub (3 repos)	0.53	0.13	0.00

Five-Dimension Rubric (rubric value justification).

The proposal specifies a 5-dimension Likert rubric (completeness, accuracy, actionability, specificity, clarity, each 1–5) [25, 50, 65]. We operationalize the rubric with Qwen2.5-3B-Instruct as the judge on a stratified subsample of 28 LLM-with-taxonomy specs (6 per issue type), calibrated against the 20 lead-author reference specs (Table 6). The LLM-with-taxonomy condition achieves a mean rubric score of 3.89/5, exceeding the hypothesized lower bound of 3.5. The per-dimension profile is: completeness 3.79, accuracy 4.39, actionability 3.71, specificity 4.07, clarity 3.50. The lead-author reference set scores lower on coverage-style dimensions (completeness 3.05, clarity 3.00) but higher on actionability (4.70 vs 3.71), an expected expert-vs-LLM contrast in which human-written specs are more developer-actionable but less verbose, confirming the judge is not biased toward LLM output.

Stage 1 Classification Recovery. The Cohen’s κ trajectory against the 490-review expert gold crosses three Landis-Koch [40] agreement bands (Table 8): the V2 LLM achieves $\kappa = 0.163$ (slight); after cleanlab correction [48] $\kappa = 0.333$ (fair); the V5 classifier retrained on the corrected labels reaches $\kappa = 0.592$ (moderate, just below the substantial

Table 8: Cohen’s κ progression vs the 490-review expert gold. Three Landis-Koch [40] bands crossed; cleanlab correction follows the confident-learning procedure of [48].

classifier	n	Cohen’s κ	Landis-Koch band
V2 LLM original	490	0.163	slight
cleanlab-corrected V2	490	0.333	fair
V5 classifier	490	0.592	moderate

Table 9: Per-class F1 against the 490-review expert gold. V2 LLM is blind to the two minority classes (compatibility, performance). The verified-anchor procedure with targeted augmentation recovers them.

class	V2 LLM	cleanlab-V2	V5
bug_report	0.43	0.50	0.71
feature_request	0.59	0.61	0.69
performance	0.00	0.47	0.77
usability	0.11	0.21	0.55
compatibility	0.00	0.00	0.83
praise	0.41	0.55	0.79
other	0.27	0.36	0.41
macro F1	0.26	0.39	0.68

threshold). On the strictly held-out 307-review subset (183 of the 490 entered training via the verified anchor), V5 reaches $\kappa = 0.616$, slightly above the 0.592 on the full 490, ruling out a contamination artifact.

The per-class recovery story (Table 9) makes the structural collapse visible. On the two minority classes the V2 LLM cannot see at all, F1 jumps to 0.83 for **compatibility** and 0.77 for **performance**. Applied as a third-opinion judge across the full 215,583-review corpus, V5 independently agrees with cleanlab on 88.66% of the 40,291 corrected cases. Two models trained without exposure to each other’s signal converging on the same fix is the strongest evidence that the corrections reflect genuine label errors rather than artifacts of the procedure. The choice of confident-learning thresholds is not load-bearing: a full 3×3 grid over (`anchor_conf`, `max_llm_prob`) $\in \{0.65, 0.70, 0.75\} \times \{0.15, 0.20, 0.25\}$ keeps V5 endorsement in an 82.7–91.0% band, with the headline (0.70, 0.20) at 88.66% sitting near the grid center.

To rule out memorization of the 200 synthetic compatibility templates used during V5 training, we evaluate V5 against Maalej’s 5,008 labels under a label-space cross-protocol test. Review texts appear in the anchor under the RRGGen seven-class taxonomy, but predictions are scored against Maalej’s separate taxonomy, which contains no **compatibility** class, making any compatibility predictions out-of-distribution with respect to the evaluation labels. On the five overlapping classes ($n = 4,560$), V5 achieves macro F1 = 0.676, weighted F1 = 0.730, and accuracy = 72.0%. V5 also flags 432 reviews (8.6%) as **compatibility**; the top-twenty-confidence flags read as canonical compatibility complaints (“crashes on iphone 5 and ipad 3”, “works on HTC Desire X but not Samsung Galaxy Tab 3”), all labeled **bug_report** in Maalej because Maalej offers no alternative

class. The detector is performing concept-recognition, not template-matching.

5.2 Is the IR Load-Bearing?

Adding the IssueSpec to RAG raises mean human-rated quality from 2.26 to 4.62 on 100 paired (`full`, `no_spec`) ratings drawn from a 400-row blinded lead-author rating set, a gain of +2.36 points (paired-bootstrap 95% CI [+2.18, +2.52], $B = 10,000$), exceeding the hypothesized minimum of one Likert point. The effect is large by every standard convention: Cliff’s $\delta = 0.882$ [11] (the “large” band in Romano et al. [54]), Cohen’s $d_z = 2.68$ paired, and Cohen’s $d_s = 3.02$ independent-sample. The matched-pair sign breakdown is 95/100 in favor of `full`, 1/100 in favor of `no_spec`, with 4 ties. The result holds across four statistical tests with non-overlapping nulls: paired Wilcoxon ($p = 1.1 \times 10^{-18}$), Friedman [16] ($\chi^2(3) = 199.3$, $p = 5.9 \times 10^{-43}$), Bradley-Terry (strength separation > 4.0 units), and McNemar (significant on 5 of 6 pairs). The simultaneous rejection under Bonferroni-corrected $\alpha = 0.05$ rules out a broader class of alternative explanations than any single test would.

The negative finding is equally important. RAG without the IssueSpec (`no_spec`, retrieving over 15,100 past replies) is statistically indistinguishable from a no-context prompt baseline at our pre-registered α : 2.26 vs 2.98, with the paired Wilcoxon failing to reject equality, validating the second half of our hypothesis. Retrieval alone does not reach paper-grade response generation in this domain. The structural intermediate carries the work; retrieval supplies style.

Vanilla RAG vs Agentic RAG. The four headline conditions are all single-shot vanilla-RAG variants by design, isolating the IssueSpec’s value from agentic orchestration. We additionally compare against the closest agentic-RAG baselines (Self-RAG [4] and DSP [37]) on a feasibility-scale demonstration: with Qwen2.5-3B as critic and reviser on $n = 10$ reviews (max two iterations), the agentic variant scores 0.70 on the rule-based scorer versus 0.58 for vanilla single-shot, and the citation rate rises from 0% to 60%. The $n = 10$ sample is not powered for inferential claims and we make no Pareto-dominance claim against the headline; the demonstration only establishes that the agentic loop is wired correctly and that the IssueSpec’s +2.36 Likert-point gain dwarfs the +0.12 rule-based gain from agentic iteration alone.

As an inter-rater check, a second human rater independently scored a 30-pair subset of the (`no_spec`, `full`) comparison and preferred `full` on 22 of 30 pairs (73.3%), with a paired-difference mean of +0.67 quality points and Cohen’s $d = 0.53$. The direction matches the lead author. Cohen’s κ on the discrete preference column is suppressed by the kappa paradox [21] (both raters cluster their preferences at the top of the Likert, inflating chance agreement); raw within-1 agreement is 86.7%.

Table 10: Experiment 2 automatic and human-rated quality on 100 paired reviews. Lead-author rated, blinded to generation method. The full system adds +2.36 quality points over `no_spec` (paired-bootstrap 95% CI [+2.18, +2.52]; Cliff’s $\delta = 0.882$ [11], Cohen’s $d_z = 2.68$ [54]); `no_spec` is statistically indistinguishable from the prompt-only baseline at Bonferroni-corrected $\alpha = 0.05$ across paired Wilcoxon, Friedman [16], Bradley-Terry, and McNemar tests.

metric	rrgen	prompt	no_spec	full
BLEU-1	0.210	0.188	0.231	0.129
BERTScore F1	0.844	0.824	0.851	0.818
quality (1-5)	2.31	2.98	2.26	4.62
helpful %	19	84	31	92

5.3 Aligning the IR-Aware Generator at Scale

Five policies were trained on distilGPT2 (82M parameters, 400 SFT samples, 296 KTO pairs, 100 DPO pairs): SFT-base, KTO [20], DPO [52], a constrained-proxy variant, and Lagrangian Constrained PPO [13, 57]. Head-to-head on a 100-review test set, per-policy BLEU-1 / ROUGE-L / BERTScore F1 are: SFT-base 0.090 / 0.097 / 0.802; KTO 0.068 / 0.074 / 0.792; DPO 0.084 / 0.088 / 0.800; **constrained-proxy 0.137 / 0.131 / 0.806**; Lagrangian PPO 0.087 / 0.091 / 0.798. The constrained proxy leads three of four metrics (BLEU-1, ROUGE-L, BERTScore F1; the rule-based safety-violation rate is uninformative here because compliance is satisfied at initialization), with +52% BLEU-1 over SFT-base. KTO underperforms here, consistent with its small-data assumption breaking on a 296-pair training set.

At distilGPT2 scale the compliance scorer is satisfied at initialization ($C = 0.94 \geq \tau$), so the Lagrange multiplier λ collapses to zero and the update reduces to single-objective optimization. This is exactly the graceful-degradation regime predicted by our hypothesis and the reason the Lagrangian PPO ties SFT-base. The contribution at this layer is the CMDP formulation, the released testbed, and validation that the loop runs end-to-end. Active-constraint validation requires an 8B-class base (Section 6.4).

5.4 Cluster Quality and Purity

Stage 2 was redesigned from a flat UMAP+HDBSCAN baseline (194 clusters, mean size 51.5) to the three-layer KG-hierarchical design (605 sub-clusters, mean size 3.0). We compute two complementary validity axes. *Intrinsic geometry*, Davies-Bouldin [14], Calinski-Harabasz [8], silhouette [55], follows standard IR/clustering practice [43, 59, 60]. *Extrinsic cluster purity* [2, 43] uses a Y/P/N weighted-purity rubric, $\text{purity}_w = (|Y| + 0.5|P|) / (|Y| + |P| + |N|)$, with Y = all 5 share theme, P = 3-4 of 5 share, N = fewer than 3 share, following inter-rater conventions compatible with Landis-Koch [40].

KG-hierarchical achieves $5.4\times$ lower Davies-Bouldin and $1.9\times$ higher Calinski-Harabasz than the flat baseline (Table 11), at the cost of finer per-cluster Y/P/N purity (0.39 vs.

Table 11: Stage 2 designs head-to-head, comparing Pure HDBSCAN (without UMAP), original flat UMAP+HDBSCAN, and KG-hierarchical. Intrinsic metrics (Davies-Bouldin [14], Calinski-Harabasz [8], silhouette [55]) follow standard IR/clustering convention [43, 59, 60]. The Y/P/N weighted-purity rubric is a discrete-coding instantiation of the extrinsic cluster-purity metric [2, 43]. The count-controlled flat baseline (agglomerative-605) is reported separately in the A1b ablation (§5.5).

metric	Pure HDB	Flat U+H	KG hier.
n_clusters	3 (deg.)	194	605
mean cluster size	~16,933	51.5	3.0
Davies-Bouldin (↓)	n/a	12.15	2.24
Calinski-Harabasz (↑)	n/a	0.98	1.85
Y/P/N purity (lead-author)	n/a	0.66	queued
Y/P/N purity (Qwen-judge)	n/a	0.625	0.390

0.625 Qwen-judge). By Landis-Koch [40] convention adapted to weighted purity, the flat baseline’s 0.66 lead-author purity sits in the substantial-agreement band (≥ 0.61) and KG-605’s 0.39 in the fair band (≥ 0.21), reflecting the granularity-vs-coherence trade-off the A1b ablation makes explicit.

Count-controlled diff from original. A count-controlled ablation (A1b) disambiguates two evaluation axes that prior cluster-quality work tends to conflate: intrinsic geometry and practitioner-side prioritization (centrality-weighted coverage). On 1,815 representative reviews (3 per sub-cluster across 605 sub-clusters), an agglomerative clustering forced to 605 clusters reaches DB = 1.12, CH = 4.21, silhouette = +0.148, against KG-605’s 2.24 / 1.85 / −0.234. The $5.4\times$ DB advantage in the headline against flat-194 is therefore primarily a cluster-count effect, not a KG-structure effect: on geometry, count-matched agglomerative clustering ties or beats the KG.

Where the KG actually wins. On the prioritization axis the story reverses. Of the 10,534 aspect nodes in the KG, the top five by PageRank account for 74% of all review-aspect mentions, and the top ten account for 96%. A developer triaging in PageRank order completes three-quarters of the complaint volume after reading five aspect buckets, whereas random aspect ordering would require thousands of buckets to reach the same coverage, a roughly $10^3\times$ concentration ratio. The KG additionally enables per-aspect drill-down, distinguishing **battery**→Samsung-drain from **battery**→fast-charge.

5.5 Ablations

The proposal specified seven ablations. Six (A1–A6) plus the reviewer-flagged A1b (count-matched flat vs KG) are resolved below; A7 (single-stream RLHF feedback) is deferred until the multi-annotator extension lands.

A1 (no KG, flat→Stage 3): Stage 3 IssueSpec quality is unchanged because Stage 3 reads cluster centroids rather than KG structure. **A1b (count-matched flat 605):** agglomerative-605 reaches DB = 1.12, CH = 4.21, silhouette

= +0.148 vs KG-605’s 2.24 / 1.85 / −0.234; the principal contribution of C3 is centrality-based prioritization rather than intrinsic-metric dominance. **A2 (no hierarchical clustering):** flat-194 Y/P/N purity = 0.625, KG-605 = 0.390; sub-cluster purity is lower at finer granularity, with the KG’s value coming from aspect drill-down plus PageRank ranking. **A3 (no taxonomy grounding):** strict template-fill drops from 0.96 to 0.69 (Table 7); without a routed template, structural metrics collapse, making type-routed templating the principal driver of Stage 3 quality. **A4 (no HITL at Stage 3):** the headline 4.62/5 already reflects pre-HITL outputs on the 100 evaluation specs (HITL-2 is a production-time gate, not applied during our benchmark), so removing HITL leaves the score unchanged. HITL’s value lies in production-time correction rather than benchmark quality. **A5 (no RAG):** with RAG removed, the system shifts by $\Delta\text{BLEU-1} = -0.006$, $\Delta\text{ROUGE-L} = -0.008$, $\Delta\text{BERTScore F1} = -0.004$ (all $< 1\%$ absolute), indicating RAG contributes negligibly to automatic overlap metrics in this benchmark. **A6 (no IssueSpec, RAG-only):** quality drops from 4.62 to 2.26 ($\Delta = -2.36$, $p < 0.001$), confirming the Stage 3→Stage 4 coupling accounts for the headline gain.

6 Discussion

6.1 Implications

Three claims travel beyond app reviews. *First*, LLM annotation noise on software-engineering text is structurally biased rather than independent-and-identically-distributed: the 25% mislabeling rate is concentrated on minority classes the LLM is blind to, not spread uniformly across the corpus. This is a finding about LLM-as-annotator failure modes that bears on any application using LLM labels for structured-text annotation. *Second*, a typed structural intermediate, not retrieval volume or agentic iteration, is what does the load-bearing work in spec-aware response generation. The +2.36 gain from adding the IssueSpec dwarfs the +0.12 gain from adding agentic critique-revise iteration to the same vanilla-RAG backbone, indicating the structural intermediate carries the work, not the iteration loop. *Third*, the count-controlled cluster-validation ablation (A1b) delivers a self-contained methodological contribution: whenever a new clustering design changes cluster count and structure simultaneously, intrinsic geometry metrics (DB, CH, silhouette) conflate the two effects. The minimum reporting standard our analysis suggests is to (a) compare against a count-matched baseline on the same intrinsic metrics, and (b) pair that with a domain-utility metric (PageRank coverage in our case). This pattern applies directly to clustering-design comparisons in IR, software engineering, and NLP.

6.2 Threats to Validity

Construct. Template-fill measures coverage rather than content correctness; SPECov is a lexical-grounding proxy [26, 32, 38, 44]. NLI- and QA-based faithfulness measurement [18, 29, 39] is a clear next step. *Internal.* The

count-versus-structure confound is exposed and resolved by the A1b ablation, which reframes C3's contribution as centrality-based prioritization rather than intrinsic geometry. *External.* RRGGen contains 58 anonymized Android apps; V5 transfers zero-shot to the Maalej label space. Stage 2–4 cross-corpus transfer remains to be validated. The headline LLM is Claude Opus, with Llama-3.3-70B and Qwen2.5- $\{3B, 1.5B\}$ as cross-family bias controls; GPT-4o and Gemini-1.5-Pro replication is queued (Section 6.4). *Statistical.* Experiment 2's four simultaneous tests reject under non-overlapping nulls and survive Bonferroni correction at $\alpha = 0.05$.

6.3 Reproducibility and Extensibility

Every numerical claim in Section 5 is backed by a saved data file and runs end-to-end from raw RRGGen to Stage 4 output in approximately six hours on a single GPU-equipped workstation. The release bundles:

- V1–V5 classifier checkpoints + cleanlab correction procedure;
- the 5,230-review verified anchor and 490-review classification gold;
- the 400-row blinded evaluation and the count-controlled A1b output;
- the SPECov extractive-coverage scorer;
- the CMDP-RLHF testbed (KTO, DPO, constrained-proxy, and Lagrangian Constrained PPO trainers);
- the 100-cluster benchmark with 99-review three-rater $\alpha = 0.451$ subsample;
- a pinned-environment Docker image (single-command build on any CUDA 12.x host).

The pipeline is composable by construction: because each stage emits a typed intermediate, components (classifier, clusterer, taxonomy template, alignment trainer) can be substituted independently without breaking downstream contracts. Future researchers can adopt or replace any single component while keeping the rest of the IR-centered evaluation harness intact. Deterministic components (clustering, classifier training) use fixed seeds; the LLM-dependent stages (annotation, IR generation, and response generation) reproduce statistically rather than bit-identically, reflecting the inherent sampling variance of the underlying models.

6.4 Future Work

Five extensions map directly to the limitations above; each starts from a resource released with this paper.

(1) **Scale the benchmark.** From 100 clusters with a 30-pair second rater to 200–300 clusters with three annotators per IssueSpec, reporting Krippendorff's α at the IssueSpec level.

(2) **CMDP RLHF on Llama-3-8B.** Where the compliance constraint is expected to bind, activating the Lagrange multiplier and making the Pareto-dominance comparison meaningful.

(3) **Frontier-family replication.** GPT-4o and Gemini-1.5-Pro on Stages 3–4. The current paper closes the Meta-family slice with Llama-3.3-70B and the Alibaba slice with Qwen2.5- $\{3B, 1.5B\}$.

(4) **Cross-corpus transfer.** Stages 2–4 on additional review corpora; V5 already transfers zero-shot to Maalej [42].

(5) **Agentic Planner extension to Stage 4.** The Planner dispatches a different sub-agent profile per issue type: defect-repair for `bug_report` via SWE-Agent [63] or RepairAgent [6], scaffolding for `feature_request`, profiler-driven optimization for `performance`, a Nielsen-heuristic checker for `usability`, and a device-OS test-matrix executor for `compatibility`. A spec-level stub already runs the dispatch across all five issue types; end-to-end patch evaluation on real repositories through HyperAgent [51], measured by patch acceptance and test-pass rate, is the next step.

7 Conclusion

We proposed **IssueSpec**, a knowledge-graph-grounded typed intermediate representation that closes the five existing research gaps identified in Section 1: classification stops at the label, response generation skips the structured intermediate, agentic software engineering assumes the spec exists, knowledge graphs emit no typed schema, and RLHF has not addressed dev-rel compliance. An isolation experiment shows the typed IR is the load-bearing component of spec-aware response generation: adding IssueSpec to RAG raises human-rated quality by +2.36 Likert points on 400 paired ratings ($p < 0.001$), while retrieval without the IR is statistically indistinguishable from a no-context baseline. A verified-anchor procedure makes the IR producible at corpus scale from noisy LLM labels, lifting Cohen's κ from 0.163 to 0.592 with 88.66% independent classifier endorsement of corrections. Cross-family replication on four LLMs (Claude, Llama, Qwen-3B, Qwen-1.5B) preserves the rank order, ruling out single-model bias. Two findings travel beyond app reviews: LLM annotation noise on software-engineering text is structurally biased rather than random, and a typed structural intermediate is what does the load-bearing work in spec-aware response generation. We release the IssueSpec schema, the verified anchor, the SPECov faithfulness metric, the count-controlled cluster-validation ablation, the structured-RAG architecture, the CMDP-RLHF trainers, and the benchmark dataset to support follow-on research.

GenAI Usage Disclosure

We disclose all uses of generative AI in this work in line with CIKM 2026 policy. *Writing and code assistance.* Anthropic's Claude (Opus 4.7) [3] assisted with grammatical refinement, caption polishing, LaTeX formatting, and reviewing implementation patterns and statistical-test code. All pipeline code, experimental scripts, and analysis were written by the human authors. Every structural, framing, experimental-design, and claim decision was made by the authors, and every reported number was produced by author-written code run under human supervision and verified

against saved data files. *Models inside the system*. Several generative models are themselves pipeline components and are reported as such in the body: Claude (Opus 4.7) is the headline Stage 3 generator (Table 7); Llama-3.3-70B-Instruct (via Groq) and Qwen2.5-{3B, 1.5B}-Instruct are cross-family Stage 3 comparators (Table 5); Qwen2.5-3B-Instruct is the LLM-as-judge for the five-dimension rubric and the Y/P/N cluster-purity audit, both calibrated against the lead author (Tables 6, 11). The V2 corpus labels in Section 5.1 are LLM-generated and explicitly treated as noisy, which is precisely what motivates the verified-anchor procedure of Stage 1. *Human verification*. No reported numerical result, expert annotation, or ground-truth label was automatically generated by AI without human verification. The 5,230-review verified anchor, the 490-review classification gold, the 99-review three-rater purity ratings, and the 20 lead-author IssueSpec references were all produced and reviewed by human annotators (Section 4).

References

- [1] Eitan Altman. 1999. *Constrained Markov Decision Processes*. CRC Press.
- [2] Enrique Amigó, Julio Gonzalo, Javier Artiles, and Felisa Verdejo. 2009. A comparison of extrinsic clustering evaluation metrics based on formal constraints. *Information Retrieval* 12, 4, 461–486.
- [3] Anthropic. 2025. Claude (Opus 4.7), Large Language Model. <https://www.anthropic.com/claude>. Used as a writing and formatting assistant during manuscript preparation; see Acknowledgments..
- [4] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *Proc. ICLR*.
- [5] Gagan Bansal, Besmira Nushi, Ece Kamar, Walter S. Lasecki, Daniel S. Weld, and Eric Horvitz. 2019. Beyond accuracy: The role of mental models in human-AI team performance. In *HCOMP*, Vol. 7. 2–11.
- [6] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. *arXiv preprint arXiv:2403.17134* (2024).
- [7] Sergey Brin and Lawrence Page. 1998. The anatomy of a large-scale hypertextual web search engine. In *Proceedings of the Seventh International Conference on World Wide Web (WWW)*. 107–117.
- [8] Tadeusz Caliński and Jerzy Harabasz. 1974. A dendrite method for cluster analysis. *Communications in Statistics* 3, 1 (1974), 1–27.
- [9] Oscar Chaparro, Jing Lu, Fiorella Zampetti, Laura Moreno, Massimiliano Di Penta, Andrian Marcus, Gabriele Bavota, and Vincent Ng. 2017. Detecting missing information in bug descriptions. In *Proc. FSE*.
- [10] Ning Chen, Jialiu Lin, Steven C. H. Hoi, Xiaokui Xiao, and Boshen Zhang. 2014. AR-miner: Mining informative reviews for developers from mobile app marketplace. In *Proc. ICSE*. ACM, 767–778.
- [11] Norman Cliff. 1993. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin* 114, 3 (1993), 494–509.
- [12] Mike Cohn. 2004. *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional.
- [13] Josef Dai, Xuehai Pan, Ruiyang Sun, Jiaming Ji, Xinbo Xu, Mickel Liu, Yizhou Wang, and Yaodong Yang. 2024. Safe RLHF: Safe Reinforcement Learning from Human Feedback. In *Proc. ICLR*. *arXiv:2310.12773*, 2023.
- [14] David L. Davies and Donald W. Bouldin. 1979. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence* PAMI-1, 2 (1979), 224–227.
- [15] A.J.W. de Vink, Natalia Amat-Lefort, and Lifeng Han. 2025. ReviewGraph: A knowledge graph embedding based framework for review rating prediction with sentiment features. In *Proc. ICKG*.
- [16] Janez Demšar. 2006. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research* 7 (2006), 1–30.
- [17] Andrea Di Sorbo, Sebastiano Panichella, Carol V. Alexandru, Corrado A. Visaggio, Gerardo Canfora, and Harald C. Gall. 2016. SURF: Summarizer of user reviews feedback. In *Proc. ICSE Companion*. IEEE.
- [18] Esin Durmus, He He, and Mona Diab. 2020. FEQA: A question answering evaluation framework for faithfulness assessment in abstractive summarization. In *Proc. ACL*.
- [19] Jacek Dąbrowski, Emmanuel Letier, Anna Perini, and Angelo Susi. 2022. Analysing app reviews for software engineering: A systematic literature review. *Empirical Software Engineering* 27, 2 (2022), 43.
- [20] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. 2024. KTO: Model Alignment as Prospect Theoretic Optimization. *arXiv preprint arXiv:2402.01306* (2024).
- [21] Alvan R Feinstein and Domenic V Cicchetti. 1990. High agreement but low kappa: I. The problems of two paradoxes. *Journal of Clinical Epidemiology* 43, 6 (1990), 543–549.
- [22] Cuiyun Gao, Jichuan Zeng, Michael R. Lyu, and Irwin King. 2019. Automating App Review Response Generation. In *Proc. ASE*. 163–175.
- [23] Cuiyun Gao, Jichuan Zeng, Michael R. Lyu, and Irwin King. 2020. CoRe: Automating review response generation with contextual encoding. In *Proc. ASE*.
- [24] Yonatan Geifman and Ran El-Yaniv. 2017. Selective classification for deep neural networks. In *NeurIPS*.
- [25] Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences (PNAS)* 120, 30 (2023).
- [26] Max Grusky, Mor Naaman, and Yoav Artzi. 2018. Newsroom: A dataset of 1.3 million summaries with diverse extractive strategies. In *NAACL-HLT*.
- [27] Emitza Guzman and Walid Maalej. 2014. How do users like this feature? A fine grained sentiment analysis of app reviews. In *Proc. RE*. 153–162.
- [28] Marti A. Hearst. 1999. Untangling text data mining. In *Proc. ACL*. 3–10.
- [29] Or Honovich, Roei Aharoni, Jonathan Herzig, Hagai Taitelbaum, Doron Kukliansky, Vered Cohen, Thomas Scialom, Idan Szpektor, Avinatan Hassidim, and Yossi Matias. 2022. TRUE: Re-evaluating factual consistency evaluation. *Proceedings of NAACL-HLT* (2022).
- [30] Mingqing Hu and Bing Liu. 2004. Mining and summarizing customer reviews. In *Proc. KDD*. 168–177.
- [31] International Organization for Standardization. 2011. ISO/IEC 25010: Systems and software engineering, Systems and software Quality Requirements and Evaluation (SQuaRE), System and software quality models. Standard.
- [32] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *Comput. Surveys* 55, 12 (2023), 1–38.
- [33] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *ICLR*.
- [34] Mona Erfani Joorabchi, Ali Mesbah, and Philippe Kruchten. 2013. Real challenges in mobile app development. In *Proc. ESEM*. 15–24.
- [35] Arvinder Kaur and Akanksha Sahu. 2023. A BERT-RCNN hybrid approach for app review classification. *arXiv preprint* (2023).
- [36] Saraswathy Keertipati, Bastin Tony Roy Savarimuthu, and Sherlock A. Licorish. 2016. Approaches for prioritizing feature improvements extracted from app reviews. In *Proc. EASE*.
- [37] Omar Khattab, Keshav Santhanam, Xiang Lisa Li, David Hall, Percy Liang, Christopher Potts, and Matei Zaharia. 2022. Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive NLP. *arXiv preprint arXiv:2212.14024*.
- [38] Wojciech Kryscinski, Bryan McCann, Caiming Xiong, and Richard Socher. 2020. Evaluating the factual consistency of abstractive text summarization. In *Proc. EMNLP*.

- [39] Philippe Laban, Tobias Schnabel, Paul N. Bennett, and Marti A. Hearst. 2022. SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. *Transactions of the Association for Computational Linguistics* 10 (2022), 163–177.
- [40] J. Richard Landis and Gary G. Koch. 1977. The measurement of observer agreement for categorical data. *Biometrics* 33, 1 (1977), 159–174.
- [41] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*.
- [42] Walid Maalej and Hadeer Nabil. 2016. Bug report, feature request, or simply praise? On automatically classifying app reviews. *Requirements Engineering* 21, 3 (2016), 311–331. doi:10.1007/s00766-016-0251-9
- [43] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press.
- [44] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. On faithfulness and factuality in abstractive summarization. In *Proc. ACL*. 1906–1919.
- [45] Leland McInnes, John Healy, and Steve Astels. 2017. HDBSCAN: Hierarchical density based clustering. *Journal of Open Source Software* 2, 11 (2017), 205.
- [46] Leland McInnes, John Healy, and James Melville. 2018. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426* (2018).
- [47] Jakob Nielsen. 1994. *Usability Engineering*. Morgan Kaufmann.
- [48] Curtis G. Northcutt, Lu Jiang, and Isaac L. Chuang. 2021. Confident Learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research* 70 (2021), 1373–1411.
- [49] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. *The PageRank citation ranking: Bringing order to the web*. Technical Report 1999-66. Stanford InfoLab.
- [50] Nicholas Pangakis, Samuel Wolken, and Nicolaus Fasching. 2023. Automated annotation with generative AI requires validation. *arXiv preprint arXiv:2306.00176* (2023).
- [51] Huy Nhat Phan, Phong X. Nguyen, and Nghi D. Q. Bui. 2024. HyperAgent: Generalist Software Engineering Agents to Solve Coding Tasks at Scale. *arXiv preprint arXiv:2409.16299* (2024).
- [52] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS*.
- [53] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proc. EMNLP*. 3982–3992.
- [54] Jeanine Romano, Jeffrey D. Kromrey, Jesse Coraggio, and Jeff Skowronek. 2006. Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys?. In *Annual meeting of the Florida Association of Institutional Research*. 1–33.
- [55] Peter J. Rousseeuw. 1987. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *J. Comput. Appl. Math.* 20 (1987), 53–65.
- [56] Sunita Sarawagi. 2008. Information Extraction. *Foundations and Trends in Databases* 1, 3 (2008), 261–377.
- [57] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [58] Faiz Ali Shah et al. 2021. Mining user opinions to support requirements engineering for mobile applications. *arXiv preprint arXiv:2108.00663* (2021).
- [59] Michael Steinbach, George Karypis, and Vipin Kumar. 2000. A comparison of document clustering techniques. In *KDD Workshop on Text Mining*.
- [60] Alexander Strehl and Joydeep Ghosh. 2002. Cluster ensembles, a knowledge reuse framework for combining multiple partitions. *Journal of Machine Learning Research* 3 (2002), 583–617.
- [61] Lorenzo Villaruel, Gabriele Bavota, Barbara Russo, Rocco Oliveto, and Massimiliano Di Penta. 2016. Release planning of mobile apps based on user reviews. In *Proc. ICSE*. ACM, 14–24.
- [62] Matt Wynne, Aslak Hellesøy, and Steve Tooke. 2017. *The Cucumber Book: Behaviour-Driven Development for Testers and Developers*. Pragmatic Bookshelf.
- [63] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *NeurIPS*.
- [64] Z. Yu, S. Wu, J. Jiang, et al. 2024. A knowledge-graph based text summarization scheme for mobile edge computing. *J. Cloud Comput.* 13 (2024), 9.
- [65] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *NeurIPS*.
- [66] Thomas Zimmermann, Rahul Premraj, Nicolas Bettenburg, Sascha Just, Adrian Schröter, and Cathrin Weiss. 2010. What makes a good bug report? *IEEE TSE* 36, 5 (2010), 618–643.

A Annotator Guidelines

The lead author (PhD in software engineering with prior mobile-app review-mining experience) served as primary annotator across all five annotation tasks. A second human rater (NLP graduate student with software-engineering background) provided inter-rater validation on a 30-pair Stage 4 subsample, and two Qwen2.5-3B-Instruct prompts (concise role-based and chain-of-thought) served as the third and fourth raters in the 99-review Krippendorff α calibration. For the 5,230-review verified anchor, annotators classified each review into one of seven Maalej-Nabil classes (`bug_report`, `feature_request`, `performance`, `usability`, `compatibility`, `praise`, `other`) with multi-label assignment permitted when a review carried multiple concerns; reviews below the “fairly confident” threshold were skipped and excluded from the anchor. For the 490-review classification gold standard, a stratified sample of 70 reviews per class was drawn from the working corpus, and the lead author re-labeled each review independently and blinded to the V2 LLM prediction; the 490 gold labels serve as the held-out evaluation set for Cohen’s κ progression (Table 8) and per-class F1 (Table 9). For the 5-dimension IssueSpec rubric ($n = 28$ specs, 6 per issue type plus 2 extras), each spec was scored on a 1–5 Likert scale across five dimensions: *completeness* (are all required template fields filled?), *accuracy* (do field values correctly reflect the source cluster?), *actionability* (would a developer know what to do?), *specificity* (does the spec name concrete entities rather than generic terms?), and *clarity* (is the spec easy to read and unambiguous?), with each dimension scored independently. For the 400-row Stage 4 response evaluation (100 reviews $\times$ 4 conditions: `rrgen`, `prompt-baseline`, `no_spec`, `full`), the lead author rated *quality* (1–5), *specificity* (1–5), and *helpful* (Y/N) while blinded to the generation method; condition labels were replaced with random A/B/C/D codes and unsealed only at aggregation, with the assignment persisted as spreadsheets with JSON master keys for audit. A second human rater independently scored a 30-pair (`no_spec`, `full`) subset for inter-rater calibration. For the Y/P/N cluster-purity rubric (initial 50-cluster audit; 99-cluster three-rater subsample), annotators read 5 representative reviews per cluster and assigned *Y* (all 5 share the same sub-theme within the cluster’s aspect), *P* (3 or 4 of 5 share the sub-theme), or *N* (fewer than 3 share); weighted purity is $\text{purity}_w =$

$(|Y| + 0.5|P|)/(|Y| + |P| + |N|)$, with Y contributing 1.0, P contributing 0.5, and N contributing 0. All comparative evaluations were blinded throughout the rating window and

unsealed only at aggregation; reviews or specs below the annotator confidence threshold were marked as “skip” and excluded from downstream analysis.